

DSA with Python

Chapter 1 · Arrays

Lists

Index

Slicing

Searching

Sorting

2D Arrays

Complexity

What is an Array?

A contiguous collection of elements accessible by index

42	17	88	5	63	29
[0]	[1]	[2]	[3]	[4]	[5]

Index →

● Ordered

Elements maintain insertion order

● 0-Indexed

First element is at index 0

● Mutable

Elements can be changed after creation

● Dynamic

Python lists grow/shrink automatically

Creating Arrays in Python

Python uses lists as dynamic arrays



```
# Empty list
arr = []

# With initial values
nums = [10, 20, 30, 40, 50]

# List comprehension
squares = [x**2 for x in range(1, 6)]    # [1, 4, 9, 16, 25]

# Using list() constructor
from_range = list(range(5))              # [0, 1, 2, 3, 4]

# Repeated elements
zeros = [0] * 5                           # [0, 0, 0, 0, 0]
matrix_row = [None] * 4                   # [None, None, None, None]
```



Python lists are heterogeneous — they can hold mixed types, but for DSA practice treat them as typed arrays.

Indexing & Slicing

Access individual elements or sub-arrays

Indexing

```
arr = [10, 20, 30, 40, 50]
arr[0]    → 10    # first
arr[-1]   → 50    # last
arr[2]    → 30    # third
arr[-2]   → 40    # second-to-last
```

Slicing `arr[start : stop : step]`

```
arr[1:4]   → [20, 30, 40]
arr[:3]    → [10, 20, 30]
arr[2:]    → [30, 40, 50]
arr[::2]   → [10, 30, 50]
arr[::-1]  → [50, 40, 30, 20, 10]
```

10

+0 / -5

20

+1 / -4

30

+2 / -3

40

+3 / -2

50

+4 / -1

⚠ *Slicing creates a new list (shallow copy) — modifications won't affect the original.*



```
arr[1:4]    # O(k) time, O(k) space (k = slice length)
```

Common Array Operations

Insert · Delete · Search · Update



```
arr = [10, 20, 30, 40]
```

— Access / Update

```
print(arr[1])           # 20                O(1)
arr[1] = 99              # [10,99,30,40]      O(1)
```

— Append / Insert

```
arr.append(50)           # [10,99,30,40,50]    O(1) amortised
arr.insert(2, 55)        # [10,99,55,30,40,50] O(n)
```

— Delete

```
arr.pop()                # removes last → O(1)
arr.pop(1)               # removes index 1 → O(n)
arr.remove(55)           # removes first occurrence → O(n)
del arr[0]               # deletes by index → O(n)
```

— Search

```
idx = arr.index(30)      # first index of 30 → O(n)
found = 30 in arr        # True / False → O(n)
cnt = arr.count(30)      # occurrences → O(n)
```

Time Complexity Cheat Sheet

Big-O for Python list operations

Operation	Method / Syntax	Time	Notes
Access by index	<code>arr[i]</code>	$O(1)$	Direct memory address
Update by index	<code>arr[i] = v</code>	$O(1)$	
Append (end)	<code>arr.append(x)</code>	$O(1)^*$	Amortised
Insert (middle)	<code>arr.insert(i, x)</code>	$O(n)$	Shifts elements right
Delete by index	<code>arr.pop(i)</code>	$O(n)$	<code>pop()</code> (end) is $O(1)$
Search (linear)	<code>x in arr</code>	$O(n)$	Unsorted list
Search (binary)	<code>bisect.bisect()</code>	$O(\log n)$	Sorted list only
Slice	<code>arr[a:b]</code>	$O(k)$	k = slice length
Length	<code>len(arr)</code>	$O(1)$	Stored in header

Linear Search

Scan every element until target is found — $O(n)$



```
def linear_search(arr, target):  
    for i, val in enumerate(arr):  
        if val == target:  
            return i    # found  
    return -1          # not found
```

```
# Example  
arr = [4, 2, 7, 1, 9, 3]  
print(linear_search(arr, 7))    # 2  
print(linear_search(arr, 5))    # -1
```

Step-through: search for 7

4

2

7

1

9

3

✓ Found at index 2

Best: $O(1)$ — first element **Worst:** $O(n)$ — last / absent **Avg:** $O(n/2) = O(n)$

Binary Search

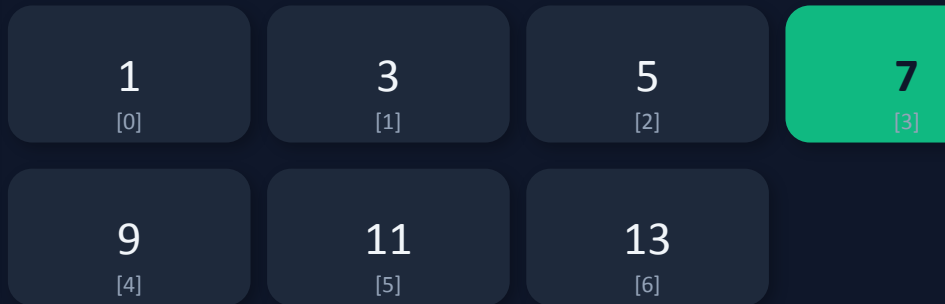
Halve the search space each step — $O(\log n)$ · Requires sorted array



```
def binary_search(arr, target):  
    lo, hi = 0, len(arr) - 1  
    while lo <= hi:  
        mid = (lo + hi) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            lo = mid + 1  
        else:  
            hi = mid - 1  
    return -1
```

```
arr = [1, 3, 5, 7, 9, 11, 13]  
binary_search(arr, 7)    # → 3
```

Searching for 7 → found in 2 steps



$mid = (0+6)//2 = 3 \rightarrow arr[3] = 7 \rightarrow \checkmark$ Found!

Time: $O(\log n)$ Space: $O(1)$ iterative / $O(\log n)$ recursive Requirement: sorted array

Sorting Algorithms

Bubble · Selection · Insertion · Python built-ins



```
# Bubble Sort  O(n²)
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
# Selection Sort  O(n²)
def selection_sort(arr):
    for i in range(len(arr)):
        min_i = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_i]:
                min_i = j
        arr[i], arr[min_i] = arr[min_i], arr[i]
```

```
# Insertion Sort  O(n²) worst, O(n) best
def insertion_sort(arr):
    for i in range(1, len(arr)):
```

Python Built-ins

```
arr.sort()           # in-place O(n log n)
sorted(arr)          # new list  O(n log n)
arr.sort(reverse=True)
sorted(arr, key=len) # custom key
```

Algorithm	Best	Avg	Worst
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Timsort (built-in)	$O(n)$	$O(n \log n)$	$O(n \log n)$

Two Pointers Technique

Reduce $O(n^2)$ to $O(n)$ using left & right pointers



```
# Two Sum (sorted array) → O(n)
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return (left, right)
        elif s < target:
            left += 1
        else:
            right -= 1
    return None

# Reverse array in-place → O(n)
def reverse_array(arr):
    l, r = 0, len(arr) - 1
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1; r -= 1
```

Two Sum: target = 16

1

4

6

9

12

15

$L \rightarrow [1] + [15] \leftarrow R = 16$ ✓

Use cases: Two Sum, palindrome check, container with most water, remove duplicates from sorted array

Sliding Window Technique

Process subarrays without redundant recomputation — $O(n)$



```
# Max sum subarray of size k → O(n)
def max_sum_subarray(arr, k):
    n = len(arr)
    if n < k:
        return None

    # First window
    win_sum = sum(arr[:k])
    max_sum = win_sum

    # Slide the window
    for i in range(k, n):
        win_sum += arr[i] - arr[i - k]
        max_sum = max(max_sum, win_sum)

    return max_sum

arr = [2, 1, 5, 1, 3, 2]
print(max_sum_subarray(arr, 3)) # 9
```

arr=[2,1,5,1,3,2] k=3

Win 1: sum=8

2

1

5

1

3

Win 2: sum=7

2

1

5

1

3

Win 3: sum=9 ←
max

2

1

5

1

3

Win 4: sum=6

2

1

5

1

3

Use cases: max/min subarray, longest substring without repeat, fixed-window averages

2D Arrays (Matrices)

Lists of lists — rows × columns



```
# Create 3×3 matrix
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Access element → O(1)
print(matrix[1][2])    # 6

# Traverse all elements → O(m*n)
for row in matrix:
    for val in row:
        print(val, end=" ")

# Create m×n matrix filled with 0
m, n = 3, 4
grid = [[0] * n for _ in range(m)]
```

Common mistake **✗** (shared refs!)

Space: $O(m \times n)$ Row traversal: $O(n)$ Full traversal: $O(m \times n)$

Matrix[row][col]

1 [0][0]	2 [0][1]	3 [0][2]
4 [1][0]	5 [1][1]	6 [1][2]
7 [2][0]	8 [2][1]	9 [2][2]

matrix[1][2] = 6 (row 1, col 2)

Kadane's Algorithm

Maximum subarray sum in $O(n)$ — classic DP on arrays



```
def max_subarray(arr):  
    max_sum = arr[0]  
    curr_sum = arr[0]  
  
    for num in arr[1:]:  
        # extend or start fresh  
        curr_sum = max(num, curr_sum + num)  
        max_sum = max(max_sum, curr_sum)  
  
    return max_sum  
  
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]  
print(max_subarray(arr))    # 6  
# Subarray: [4, -1, 2, 1]
```

Trace curr_sum / max_sum

	-2	1	-3	4	-1
curr:	-2	1	-2	4	3
max:	-2	1	1	4	4

Max subarray = [4, -1, 2, 1] → sum = 6

Key insight: if curr_sum goes negative, it hurts any future subarray — reset by starting fresh at the current element.

Python Array Toolkit

Built-ins, tricks & the array module



```
import array # typed C-array (faster than list for pure numbers)
a = array.array('i', [1, 2, 3, 4]) # 'i' = signed int
```

— Useful built-ins —

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]
print(min(nums), max(nums)) # 1 9 O(n)
print(sum(nums)) # 31 O(n)
print(sorted(nums)) # [1,1,2,3,4,5,6,9] O(n log n)
```

— Enumerate & zip —

```
for i, v in enumerate(nums): # gives (index, value) pairs
    pass
```

```
for a, b in zip([1,2,3], [4,5,6]): # parallel iteration
    pass
```

— List comprehensions —

```
evens = [x for x in nums if x % 2 == 0] # filter
```

copy()

`arr.copy()` for shallow clone — never use `b=a` for arrays

Deque

`from collections import deque` for $O(1)$ left-append

bisect

`import bisect` for $O(\log n)$ sorted insert/search

heapq

`import heapq` for $O(\log n)$ priority queue on a list

Practice Problems

Ordered by difficulty — start from Easy

Easy

 Two Sum (LeetCode #1)

 Best Time to Buy & Sell Stock (#121)

 Contains Duplicate (#217)

 Move Zeroes (#283)

Medium

 Maximum Subarray — Kadane's (#53)

 Product of Array Except Self (#238)

 Rotate Array (#189)

 3Sum (#15)

Hard

 Trapping Rain Water (#42)

 First Missing Positive (#41)

 Median of Two Sorted Arrays (#4)

 Maximum Sliding Window (#239)

Tip: solve each problem with brute force first, then optimise with two-pointer / sliding window / binary search.

Chapter 1 Recap



Arrays / Lists

Dynamic, 0-indexed, $O(1)$ access



Indexing & Slicing

`arr[i]`, `arr[a:b:step]`, negative indices



Linear Search

$O(n)$ — works on unsorted arrays



Binary Search

$O(\log n)$ — sorted arrays only



Sorting

Bubble/Selection/Insertion $O(n^2)$ · Timsort $O(n \log n)$



Two Pointers

Shrinks $O(n^2)$ problems to $O(n)$



Sliding Window

Fixed/variable window for subarray problems



Kadane's Algo

Max subarray sum in $O(n)$ — greedy DP